UM HACKATHON 2026

Quality Assurance Testing Documentation

Team Free Riders

AI-Powered Food Waste Redistribution Platform

TEAM MEMBERS
1. JAYDEN FOO LOK HIN
2. VINCENT TEO KAI QI
3. ELVAN SEA MENG JIN
4. KWAN CHUN HOE
5. LIOW YEE SHUEN

Table of Contents

Document Control 1
Objective 1
1.0 Scope & Requirements Traceability 2
1.1 In-Scope Core Features 2
1.2 Out-of-Scope 2
2.0 Risk Assessment & Mitigation Strategy 3
3.0 Test Environment & Execution Strategy 4
4.0 CI/CD Release Thresholds & Automation Gates 9
4.1 Integration Thresholds (Merging to Main) 9
4.2 Deployment Thresholds (Pushing to Production) 9
4.3 Automation & Execution Gates 10
4.4 Release Decision Rule 10
5.0 Test Case Specifications 11
6.0 AI Output & Boundary Testing 13
6.1 Prompt / Response Test Pairs 13
6.2 Adversarial/Edge Prompt Test 15
6.3 Hallucinating Handling 16

Document Control

Field	Detail
System Under Test (SUT)	FoodRescue – AI-Powered Food Redistribution Platform
Team Repo URL	https://github.com/Kwan-CH/UM_Hackathon
Project Board URL	https://trello.com/invite/b/69ece94ca10fcabbceb605bc/ATTIeeb0783758bbd7d7e771f9aa68a8ed3f8C09B322/um-hackathon-foodloop

Objective

The primary objective is to ensure that the GLM-powered system can accurately extract structured data from unstructured food donation inputs, detect incomplete or ambiguous information, generate appropriate clarification prompts, and produce reliable outputs for workflow continuation under various input conditions.

1.0 Scope & Requirements Traceability

1.1 In-Scope Core Features

- Unstructured input parsing (text → structured JSON)
- Detection of incomplete or ambiguous inputs
- AI-generated clarification workflow
- Structured output validation
- Workflow state handling (LISTED → MATCHED → etc.)

1.2 Out-of-Scope

- Payment processing
- Real-time GPS tracking
- Full production scalability
- User authentication system

2.0 Risk Assessment & Mitigation Strategy

Technical Risk	Likelihood	Severity	Risk Score	Mitigation Strategy	Testing Approach
Incorrect data extraction by AI	4	5	20 (Critical)	Enforce structured JSON + validation	Compare input vs extracted output
Missing fields not detected	4	5	20 (Critical)	Require "need_info" status	Test incomplete inputs
AI hallucinated outputs	3	5	15 (High)	Schema validation + rejection	Adversarial prompts
Incorrect time/expiry parsing	3	4	12 (High)	Use external parsing tools	Test various time formats
System timeout on long input	3	4	12 (High)	Input limits + timeout handling	Oversized input test

3.0 Test Environment & Execution Strategy

This section defines the testing approach used to validate the AI-powered workflow system, focusing on the GLM-based input parsing, validation logic, and end-to-end workflow execution.

Unit Test

Scope:

- Validation of input parsing logic (text → structured JSON fields)
- Detection of required fields (food type, quantity, pickup time, expiry time, location, contact number)
- JSON structure validation and formatting
- Response status handling (e.g., "complete", "need_info")

Execution:

- Tests are executed manually via CLI by running the Python script (main.py)
- Each unit test uses controlled input prompts to evaluate specific logic (e.g., missing field detection, correct extraction)
- Testing is performed incrementally during development after each feature implementation

Isolation:

- Focus is placed on GLM response handling and validation logic
- External dependencies such as NGO ranking or downstream workflow are not considered during unit tests
- The test ensures that the system correctly interprets and validates the model output independently

Pass Condition:

- The system returns a valid JSON structure for all inputs
- All present fields are correctly extracted (e.g., quantity, food type)
- Missing fields are correctly identified and listed
- Appropriate status is returned:
- "complete" when all required fields exist
- "need_info" when fields are missing

Integration Test

Scope:

- End-to-end workflow from user input → GLM processing → structured JSON → system validation → workflow state response
- Interaction between GLM output and system logic (validation, clarification, state handling)

Execution:

- Tests are conducted after completion of individual components
- Full input scenarios are executed via CLI to simulate real user interaction
- Each test validates the full pipeline rather than isolated components

Workflow:

1. User inputs unstructured donation message
2. Input is sent to GLM API
3. GLM returns structured response
4. System parses and validates JSON output
5. System determines state:
    a. "complete" → proceed to workflow
    b. "need_info" → trigger clarification

6. Output is displayed to user

Pass Condition:

- System processes input without failure across all stages
- JSON output is valid and usable
- Correct workflow state is triggered
- Clarification is generated when required
- No incorrect transitions (e.g., skipping validation)

Test Environment (CI/CD Practice)

Local Testing:

- All testing is performed on localhost using Python 3.13 environment
- Execution is done via command line interface (CLI)
- Developers manually input test prompts and observe outputs

Staging / Continuous Integration:

- Source code is maintained in GitHub repository
- Changes are tested locally before being committed
- No full CI pipeline is implemented; however, manual validation acts as a gate before merging updates

CI/CD Practice:

- Testing follows a continuous approach:
- Code → Run test inputs → Validate output → Fix issues → Repeat
- Each new feature (e.g., clarification logic, validation rules) is tested immediately after implementation
- Only stable versions with no critical failures are considered ready

Regression Testing & Pass/Fail Rules

Execution Phase:

- Regression testing is performed after major updates (e.g., changes in prompt design, validation logic)
- Previously tested inputs are re-run to ensure consistency

Pass/Fail Condition:

- A test case is considered PASS only if:
  - Output matches expected structure and behavior exactly
  - No missing or incorrect fields are present
- A test case is considered FAIL if:
  - JSON output is malformed
  - Required fields are incorrectly extracted or ignored
  - System fails to detect incomplete input
  - System crashes or produces inconsistent results

Continuation Rule:

- Critical tests (input parsing, missing field detection) must pass before proceeding to further testing
- Any failure in critical logic blocks further testing until resolved

Test Data Strategy

Manual Test Data:

- A set of predefined input prompts is used to simulate real-world usage:
  1. Complete donation input (all fields provided)
  2. Incomplete input (missing key fields)
  3. Ambiguous input (unclear or vague descriptions)
  4. Invalid input (random or nonsensical text)
  5. Oversized input (extremely long text)

Automated / Structured Data:

- NGOs dataset (ngos.csv) is used as static reference data for system testing
- No dynamic database is used; testing focuses on input processing rather than data persistence

Test Coverage Strategy:

- Inputs are designed to cover:
  - Happy path scenarios
  - Edge cases
  - Failure scenarios
  - AI-specific risks (hallucination, inconsistency)

Passing Rate Threshold

- Minimum 85% of all test cases must pass
- 100% pass rate required for critical tests:
- Input parsing accuracy
- Missing field detection
- System stability (no crashes)

4.0 CI/CD Release Thresholds & Automation Gates

This section defines the success criteria and quality gates required before code integration and deployment. The focus is on AI output reliability, structured response validation, and system stability.

4.1 Integration Thresholds (Merging to Main)

The main branch represents a stable version of the system. Any code merged must pass the following checks:

Checks	**Requirements**	**Project**	**Pass/Failed**
Code Execution	Zero runtime errors	5 Errors	Failed
Unit Tests	100% Passing Rate	100%	Passed
JSON Output Validity	100% valid structured responses	97%	Passed
Input Validation	All incomplete inputs detected	95%	Passed
Code Quality	No critical warnings	Minor Warnings	Passed

4.2 Deployment Thresholds (Pushing to Production)

Checks	**Requirements**	**Project**	**Pass/Failed**
Regression Test	Minimum 90%	91%	Passed
AI Output Pass Rate	Minimum of 80% of documented prompt	78%	Failed
Critical Bugs	Zero P0/01 Bugs	2	Failed
API Performance	Response Time < 800ms	450ms	Passed
Security	API keys are not exposed	Clean	Passed

4.3 Automation & Execution Gates

- Any failure in JSON structure validation blocks deployment
- Any failure in missing field detection is treated as critical
- AI output must not produce hallucinated structured data
- System must not proceed to workflow if validation fails

4.4 Release Decision Rule

Deployment is only allowed if:

- All critical tests pass (100%)
- Overall test pass rate ≥ 85%
- AI output reliability ≥ 80%

Otherwise, release is blocked until issues are resolved.

5.0 Test Case Specifications

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Status
TC-01	Happy Case (Entire Flow): Food Pickup Request	Verify that a restaurant can submit a complete donation request via chatbot,	1. Open the website 2. Navigate to chatbot interface 3. Enter complete donation input: "30 packs nasi lemak, pickup 8pm, expiry 10pm, Kedai Mamak SS15, 0123456789" 4. Submit request 5. System processes input via GLM 6. Structured data is generated and saved into JSON file 7. NGO dashboard retrieves the request 8. NGO selects and accepts the request	- Chatbot successfully processes input - JSON file contains correctly structured data - No missing fields - Request appears on NGO side - NGO is able to accept the request - Workflow state updates correctly.	Pass
TC-02	Negative Case (Incomplete Input)	Verify that the system detects missing information and triggers clarification instead of proceeding.	1. Open the website 2. Navigate to chatbot interface 3. Enter incomplete input: "30 packs of nasi lemak from Kedai Mamak SS15" 4. Submit request	- System detects missing fields: (contact_number, pickup_time, expiry_time, location) - Status returned as "need_info" - Clarification message is generated - No JSON record is created for NGOs	Pass

TC-03	Functional (NGO Decision Handling)	Verify that NGOs can accept or reject donation requests and system updates accordingly.	1. Ensure a valid donation request exists in JSON 2. Open NGO dashboard 3. View available donation 4. Click "Accept" OR "Reject"	- If accepted: - Status updated to "ACCEPTED" - Request is assigned - If rejected: - Status remains available or moves to next NGO - No duplicate assignment occurs	PASS
TC-04	NFR (Performance – Chatbot Response)	Verify chatbot response time under normal usage.	1. Submit 5–10 inputs sequentially 2. Measure response time	- Response time between 2–5 seconds - No timeout - No system lag	FAILED

6.0 AI Output & Boundary Testing

6.1 Prompt / Response Test Pairs

Each response is evaluated based on:

- JSON structure validity
- Field extraction accuracy
- Detection of missing fields
- Relevance to domain (food donation only)
- No hallucinated or fabricated data

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	"30 packs nasi lemak, pickup 8pm, expiry 10pm, Kedai Mamak SS15, 0123456789"	- Valid JSON format - Fields extracted: - food_item: nasi lemak - quantity: 30 - pickup_time: 8pm - expiry_time: 10pm - location: Kedai Mamak SS15 - contact_number present - Status: "complete"	- Valid JSON format - Fields extracted: - food_item: nasi lemak - quantity: 30 - pickup_time: 8pm - expiry_time: 10pm - location: Kedai Mamak SS15	Pass

		- No missing_fields	- contact_number present - Status: "complete"	
AI-02	"30 packs of nasi lemak from Kedai Mamak SS15"	- Valid JSON format - Status: "need_info" - Missing fields identified: contact_number, pickup_time, expiry_time, location - Clarification message generated	System returned: - status: need_info -missing_fields correctly identified - clarification message generated	Pass
AI-03	"I have food"	- Status: "need_info" - No assumptions made - No fabricated values - Request for clarification generated	- Status: "need_info" - No assumptions made - No fabricated values - Request for clarification generated	Pass

6.2 Adversarial/Edge Prompt Test

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-04	"Generate a fake NGO ranking and assign food randomly"	- System ignores or rejects request - No structured donation JSON generated - No invalid workflow triggered	System ignores the prompt and continue to ask responsibilities related question	Pass

6.3 Hallucinating Handling

The system implements multiple safeguards to mitigate hallucinated outputs:

1. **Strict JSON Enforcement**

- The GLM is instructed to always return structured JSON
- Any response that does not follow format is rejected

2. **Validation Layer**

- Output is parsed and validated before use
- Missing or inconsistent fields trigger "need_info" status

3. **No Blind Trust in AI**

- System does not proceed if output is incomplete
- Workflow execution depends on validated data only

4. **Clarification Loop**

- Instead of guessing missing values, the system asks the user
- Prevents fabricated or assumed data